Certificados de Seguridad SSL (Secure Sockets Layer)

¿Qué es SSL, para que sirve y cómo funciona?

Un certificado SSL (Secure Sockets Layer, hoy en día basado técnicamente en el protocolo TLS) es un archivo digital que autentica la identidad de un sitio web y cifra la conexión entre el servidor y el navegador del usuario.

- **Cifrado de datos**: Protege la información sensible (contraseñas, datos personales y números de tarjetas de crédito) para que no sea interceptada por terceros.
- **Autenticación**: Garantiza a los visitantes que están en el sitio web real y no en una página falsa.
- **Confianza y SEO**: Elimina la advertencia de "No seguro" en los navegadores y muestra el ícono de candado junto a la URL (activando el protocolo HTTPS).

El navegador envía al servidor la versión de TLS compatible, lista de algoritmos cifradores (cipher suites) y un número aleatorio.

El servidor elige los parámetros de cifrado y responde enviando su **certificado digital** (que incluye su clave pública) firmado por una Autoridad Certificadora (CA).

El navegador verifica que el certificado sea válido y emitido por una CA de confianza.

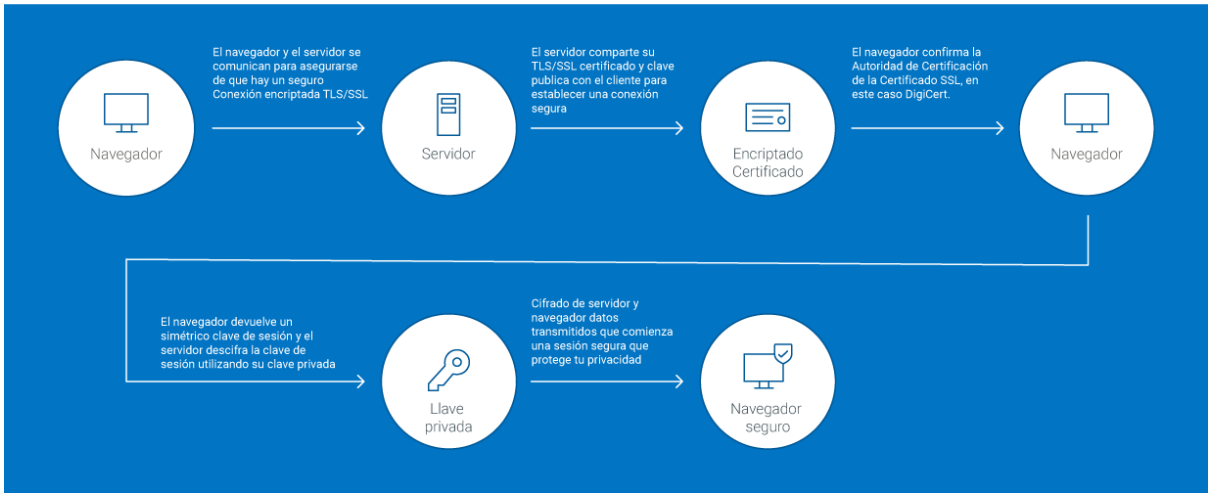
El cliente genera una clave pre-maestra, la cifra con la clave pública del servidor y se la envía. Ambos extremos derivan una clave simétrica de sesión.

A partir de este momento, todo el tráfico HTTP se transmite cifrado con la clave simétrica (más rápida y eficiente que la asimétrica).

¿Cómo se identifica visualmente un navegador que se está utilizando SSL?

- URL con https://: La dirección comienza por https:// (HTTP Secure) en lugar de http://.
- Icono de candado: Aparece un candado cerrado junto a la barra de direcciones.
- Detalles del certificado: Al hacer clic en el candado, el navegador despliega el estado de la conexión ("Conexión segura") y permite examinar el emisor del certificado, la entidad propietaria y el periodo de validez.
- Avisos de sitio no seguro: Si un sitio usa HTTP o tiene un certificado vencido/inválido, el navegador muestra la etiqueta explícita "No es seguro" o bloquea el acceso con una pantalla de advertencia roja.

Realizar una gráfica del funcionamiento.



Fuente: https://www.digicert.com/es/how-tls-ssl-certificates-work

¿Cuál es la diferencia entre HTTP y HTTPS?

Característica	**HTTP (HyperText Transfer Protocol)**	**HTTPS (HTTP Secure)**
Seguridad	Texto plano (sin cifrar). Vulnerable a la intercepción (*Sniffing*).	Cifrado mediante TLS/SSL. Protegido contra ataques *Man-in-the-Middle*.
Puerto predeterminado	Puerto 80.	Puerto 443.
Autenticación	No valida la identidad del servidor.	Valida la identidad mediante certificados digitales emitidos por CAs.
Impacto en SEO y Navegación	Penalizado por buscadores; marcado como "No seguro".	Requisito estándar; priorizado por motores de búsqueda como Google.

¿Cómo puede obtener un certificado SSL gratuito?

Se puede obtener un certificado SSL gratuito a través de varias autoridades certificadoras y herramientas confiables que permiten activar el protocolo HTTPS en tu sitio web.

- Let's Encrypt: Autoridad Certificadora abierta y gratuita ampliamente utilizada. Permite automatizar la emisión y renovación mediante herramientas como Certbot en servidores Apache, Nginx o Linux.
- Cloudflare: Proporciona certificados SSL/TLS gratuitos mediante su servicio de Proxy y DNS administrado.
- ZeroSSL: Ofrece emisión gratuita de certificados SSL compatibles con ACME.

- Plataformas de Despliegue: Servicios como Vercel, Netlify, AWS ACM o Render emiten y gestionan certificados SSL gratuitos automáticamente al vincular un dominio.

Json Web Token (JWT):

¿Qué es JWT (Json Web Token) y cómo funciona?

Un JWT (JSON Web Token) es un estándar abierto (RFC 7519) que define una forma compacta y autónoma para transmitir información de forma segura entre dos partes (como el cliente y un servidor) como un objeto JSON.

Se utiliza principalmente para la autenticación y la autorización en aplicaciones web y API.

El flujo típico de autenticación con JWT sigue estos pasos:

1. Inicio de sesión: El usuario envía sus credenciales (nombre de usuario y contraseña) al servidor.

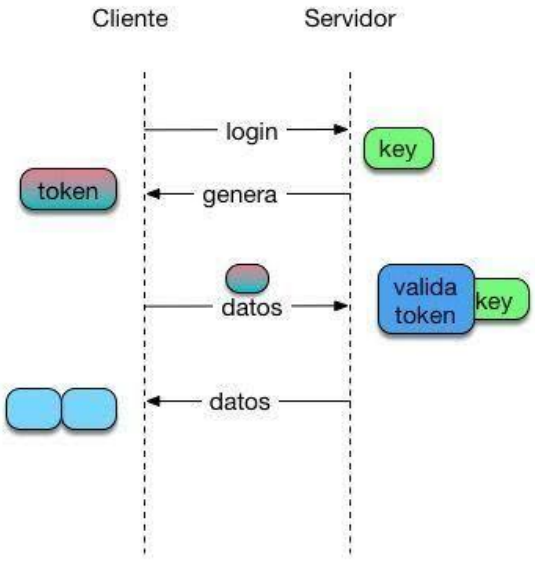
2. Validación y emisión: El servidor verifica las credenciales, genera un JWT firmado con su clave secreta y se lo devuelve al cliente (navegador o app móvil).

3. Almacenamiento en el cliente: El cliente guarda el token (generalmente en memoria, cookies seguras o almacenamiento local).

4. Envío de peticiones: En cada petición posterior a rutas protegidas, el cliente envía el JWT en la cabecera HTTP de la petición usando el esquema Bearer (Authorization: Bearer <token>).

5. Verificación: El servidor recibe la petición, toma la clave secreta para comprobar la firma del token y revisa que no haya expirado. Si todo es válido, procesa la solicitud sin necesidad de consultar una base de datos para saber quién es el usuario.

Realizar una gráfica del funcionamiento.



Fuente: https://www.arquitecturajava.com/introduccion-a-json-web-token/

AUTH0:

¿Qué es AUTH0 y cómo funciona?

Es una plataforma de Gestión de Identidades y Acceso (IAM) que funciona como un servicio de autenticación y autorización (Authentication as a Service o AaaS). Permite a los desarrolladores verificar la identidad de los usuarios y controlar el acceso a sus aplicaciones o sitios web sin necesidad de programar todo el sistema de seguridad desde cero.

- Servicio externo (AaaS): Externaliza la lógica de login, contraseñas y base de datos de usuarios a una plataforma especializada.
- Compatibilidad universal: Se puede integrar en aplicaciones móviles, web o de servidor escritas en cualquier lenguaje o framework usando sus SDKs.
- Seguridad avanzada: Ofrece protecciones integradas contra ataques comunes, filtraciones de credenciales y cumplimiento de normativas del sector.

El funcionamiento general se basa en un flujo centralizado de tokens:

1. Intento de acceso: El usuario abre tu aplicación e intenta iniciar sesión. Si no está autenticado, la aplicación lo redirige a la pantalla de login gestionada por Auth0 (conocida como Universal Login).

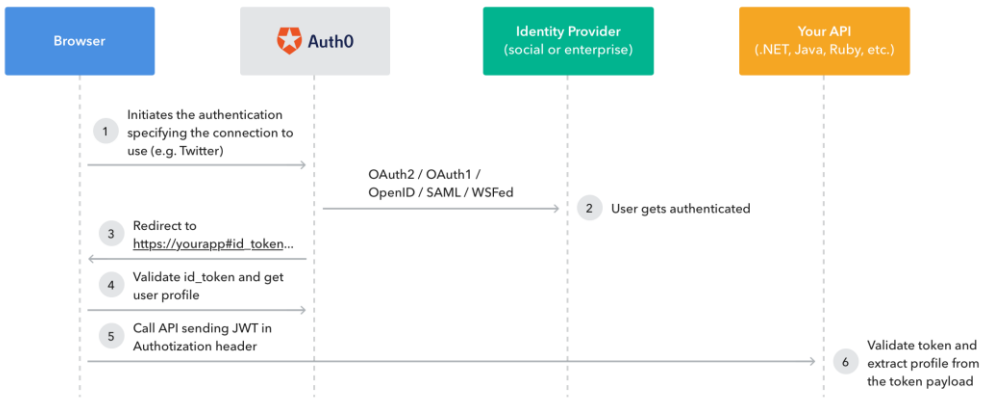
2. Verificación de identidad: El usuario introduce sus credenciales (correo y contraseña, huella biométrica, código por SMS, o mediante redes sociales como Google, GitHub o Microsoft).

3. Emisión de tokens: Auth0 comprueba la identidad y, si es correcta, emite un token (como un JWT o JSON Web Token) que contiene información de la sesión y los permisos del usuario.

4. Regreso a la aplicación: El usuario es redirigido de vuelta a tu aplicación con ese token.

5. Validación del acceso: Tu aplicación valida la firma del token y le concede al usuario el acceso correspondiente a las vistas o a las APIs protegidas.

Realizar una gráfica del funcionamiento.



Fuente: https://auth0.com/es/learn/how-auth0-uses-identity-industry-standards

API´s Seguras:

¿Cuál es la importancia de que nuestras API's sean seguras?

Las APIs constituyen la puerta de entrada a la lógica de negocio y las bases de datos de una aplicación. No asegurar una API expone la infraestructura a:

- **Fuga de datos sensibles:** Acceso no autorizado a información personal (PII), financiera o de credenciales.
- **Ataques automatizados:** Abuso por parte de bots, intentos de fuerza bruta y ataques de Denegación de Servicio (DDoS).
- **Vulneración de la integridad del sistema:** Manipulación indebida de estados o recursos del sistema mediante inyecciones o elevación de privilegios.
- **Incumplimiento normativo:** Sanciones legales por violación de regulaciones de privacidad y protección de datos (ej. GDPR, PCI-DSS).

Detallar las buenas prácticas de programación, respecto de seguridad, que se deben tener en cuenta al momento de desarrollar una API.

- Autenticación y Autorización Robustas:
    - Utilizar OAuth 2.0 / OpenID Connect con tokens JWT de corta duración.
    - Aplicar control de acceso basado en roles o atributos (RBAC / ABAC) verificando los permisos (scopes) en cada endpoint.
    - Utilizar identificadores no predecibles (como UUID v4) en las rutas para prevenir vulnerabilidades de tipo BOLA/IDOR (Broken Object Level Authorization).

- Cifrado Obligatorio en Tránsito y Reposo:
    - Forzar el uso exclusivo de HTTPS (TLS 1.2 / TLS 1.3) en todos los entornos.
    - Implementar la cabecera Strict-Transport-Security (HSTS).
    - Cifrar información sensible en la base de datos usando algoritmos seguros (ej. Argon2 o bcrypt para contraseñas; AES-256 para datos en reposo).

- Validación y Sanitización Estricta de Entradas (Input Validation):
    - Validar el tipo de dato, formato, longitud y estructura de cada parámetro de entrada o payload JSON/XML mediante esquemas estrictos (ej. DTOs con Bean Validation, Zod, Joi).
    - Utilizar consultas preparadas u ORMs configurados correctamente para prevenir Inyección SQL / NoSQL.

- Limitación de Tasa (Rate Limiting) y Throttling:

    - Restringir la cantidad de peticiones permitidas por dirección IP o por usuario en un intervalo de tiempo (ej. usando Redis o un API Gateway como Nginx o Kong) para prevenir ataques DDoS y de fuerza bruta.

- Manejo Seguro de Errores y Logs:
    - Evitar exponer la traza del código (stack traces), detalles de la base de datos o versiones del servidor en los mensajes de error devueltos al cliente.
    - Retornar respuestas estándar con códigos de estado HTTP adecuados (400, 401, 403, 404, 500).
    - Registrar eventos de seguridad relevantes (errores de autenticación, accesos denegados) sin almacenar datos sensibles o tokens en los logs.

- Configuración Correcta de CORS y Encabezados de Seguridad:
    - Configurar CORS (Cross-Origin Resource Sharing) especificando explícitamente los dominios autorizados en Access-Control-Allow-Origin, evitando el uso del comodín * en APIs autenticadas.
    - Agregar cabeceras de protección como X-Content-Type-Options: nosniff y X-Frame-Options: DENY.

- Gestión Adecuada de Secretos:
    - Nunca incluir claves de API, secretos de firma o credenciales directamente en el código fuente o en control de versiones (Git).
    - Administrar la configuración mediante variables de entorno (.env) o gestores de secretos dedicados (como HashiCorp Vault o AWS Secrets Manager).